SYSTEM REFERENCE MANUAL

Agentic RAG

An end-to-end picture of the running system: how a question becomes an answer, how the seven AI agent modes decide and retrieve, and how production layers (guardrails, cache, metrics, deploy) wrap that intelligence.

> **What this document is for.** Keep it beside the repo. It is written from the actual code paths (`src/runner.py`, `src/graph/*`, `src/api/server.py`, ingestion/retrieval, Docker Compose) so you can re-orient quickly months later without re-deriving the architecture from memory.

Edition: August 2026
Scope: Full stack — agents, retrieval, API, frontend, ops
Default mode: `agentic`
Canonical entry: `run_agent()` / `stream_agent()`
Stack: LangChain · LangGraph · FastAPI · Chroma · OpenAI · Redis · React

Contents

1. How to read this document

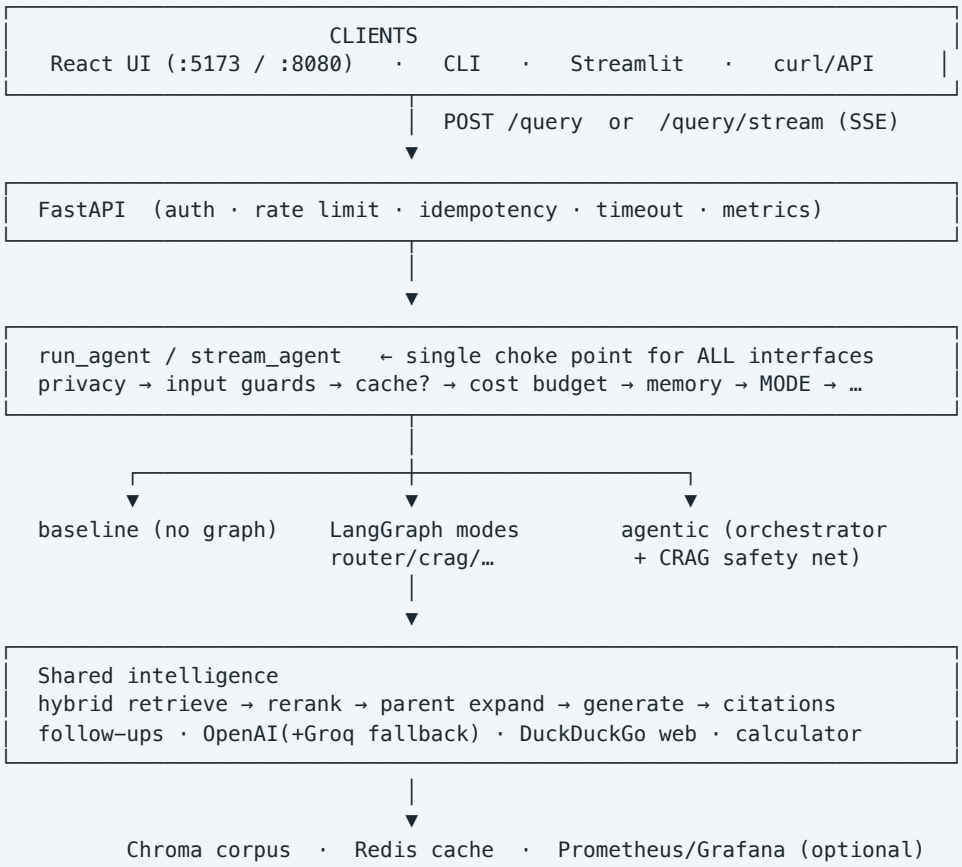
> Start with §2–§4 for the whole picture. Spend the most time on §5–§6 — that is the AI agent system. Use §7–§11 when debugging ingestion, retrieval, or production. §13 has concrete walkthroughs you can re-run with the CLI.

If you need…	Go to
One diagram of the whole product	§2 System at a glance
What happens on every API/CLI call	§4 Request lifecycle
How each agent mode thinks and routes	§5 AI agent system
Why `agentic` picked decompose vs multi-hop	§5.7 and §6
How PDFs become searchable chunks	§7 Ingestion & retrieval
Auth, rate limits, cache, metrics, Docker	§9–§10

> **STABILITY NOTE**
> Mode keys (`baseline` … `agentic`), graph node names, and the `run_agent` choke point are the durable contracts. Prompt text and default model names may change; treat them as tunable, not structural.

2. System at a glance

Agentic RAG is a **multi-mode research assistant**. One question can be answered seven different ways — from a fixed retrieve→generate baseline up to a full orchestrator that chooses strategy, grades evidence, and may rewrite or fall back to the web. Every path shares the same retrieval stack, citations, guardrails, and optional Redis cache.



Mode key	Phase	One-line behavior
`baseline`	1	Always retrieve → generate. No decisions.
`router`	2	Route to direct / retrieve / web.
`crag`	3	Grade docs; rewrite & retry; else web fallback.
`decompose`	4	Split into sub-queries; retrieve in parallel; synthesize.
`multi_hop`	5	Sequential hops; each query depends on prior findings.
`tools`	6	Function-calling loop: retrieve / web / calculator.
`agentic`	7	Router → strategy picker → subgraph → grade → answer.

3. Layered architecture

Think in five layers. Higher layers never bypass lower safety layers.

Layer	Responsibility	Primary modules
1. Interface	HTTP, CLI, UI. Auth headers, SSE framing, session IDs.	`src/api/server.py`, `frontend/`, `src/cli.py`
2. Control plane	Privacy, input/output guards, cost budgets, cache, memory packing.	`runner.py`, `guardrails.py`, `privacy.py`, `cache/`
3. Agent graphs	Mode-specific LangGraph StateGraphs (or baseline pipeline).	`src/graph/*`, `src/rag/baseline.py`, `src/agents/*`
4. Knowledge plane	Ingest, hybrid search, rerank, parent expansion, citations.	`src/ingestion/*`, `src/retrieval/*`
5. Model & tools	LLM client, structured outputs, tools, web search.	`src/llm.py`, `src/chains/`, `src/tools/`

> **HARD RULE**
> Never call `ask_baseline`, `ask_crag`, etc. from a new interface. Always go through `run_agent()` or `stream_agent()`. Skipping the runner skips privacy, rate/cost guards, cache, and follow-ups.

4. End-to-end request lifecycle

4.1 Synchronous path — `POST /query` → `run_agent()`

1. **API gate** — API key (if required), per-client rate limit, optional `Idempotency-Key`, request timeout.
2. **Privacy (input)** — block if disallowed PII/PHI patterns match policy.
3. **Input guardrails** — length, word count, credential-shaped strings (not a keyword blocklist).
4. **Answer cache** — if `CACHE_ENABLED` and no memory-bearing history: lookup Redis by normalized `question` + `mode`.
5. **Cost / rate budget** — process-wide queries-per-minute and token budgets.
6. **Memory augment** — pack recent Q+A (+ older questions only) into the effective question when `use_memory` is on.
7. **Mode dispatch** — `_dispatch(mode)` → the selected graph or baseline pipeline; token usage recorded via OpenAI callback.
8. **Output guardrails + privacy** — validate answer/sources; redact or block output PII per settings.
9. **Optional quality check** — only if `QUALITY_GUARDRAILS_ENABLED=true` (extra LLM calls).
10. **Follow-ups** — up to 3 grounded questions attached (never fails the request).
11. **Cache write** — store serializable response (not full `context_docs`) when eligible.
12. **API response** — `QueryResponse` with answer, citations, steps, follow-ups, latency, session_id.

4.2 Streaming path — `POST /query/stream` → `stream_agent()`

Same preflight as sync. During the graph run, a ContextVar emitter (`src/streaming.py`) pushes events. The React UI uses this by default.

SSE `type`	Meaning
`step`	Agent/graph progress line (router decision, hop, grade, …)
`token`	Final-answer token as it generates
`answer`	Full answer text (after generation / post-guards)
`follow_ups`	List of follow-up questions
`sources`	Source labels + `citations` array
`done`	Terminal frame: latency, session, mode, route, steps
`error`	Guardrail / runtime failure

> **CACHE HIT BEHAVIOR**
> On cache hit, streaming still emits `step` / `answer` / `follow_ups` / `sources` / `done` (with `cached: true`) so the UI path stays identical — without re-running the LLM.

5. The AI agent system

> This section is the heart of the product. Each mode is a different control policy over the same knowledge plane. Node names below match the compiled LangGraph graphs in `src/graph/`.

What every mode returns

An `AgentResponse`: `answer`, `mode`, `sources`, `citations`, `steps`, optional `route` / `grade_summary` / `sub_queries`, and `follow_ups` (attached by the runner).

How to choose a mode

Use `baseline` / `router` for speed; `crag` for quality; `decompose` / `multi_hop` for structure; `tools` for mixed tools; `agentic` when you want the system to pick.

MODE 5.1 `baseline` — Fixed pipeline

File: `src/rag/baseline.py` · **Not a LangGraph.** Always retrieves, always generates. The control group for every agentic mode.

```
question → retrieve(hybrid→rerank→top_k) → format_docs → rag_chain → AgentResponse
```

Use when: simple factual questions; latency-sensitive traffic; A/B baseline.

MODE 5.2 `router` — Intent routing

File: `src/graph/router_graph.py` · **Decision:** `router_chain` → `direct` | `retrieve` | `web_search`

```
START → classify ─┬─ direct ──────────→ direct_answer → END
                  ├─ retrieve ──→ retrieve → generate → END
                  └─ web_search → web_search ──────────→ END
```

Use when: greetings and chitchat should skip retrieval; newsy questions need the web.

MODE 5.3 `crag` — Corrective RAG

File: `src/graph/crag_graph.py` · Adds a **grader loop** after retrieval. Largest quality jump over baseline.

```
START → classify ─┬─ direct → direct_answer → END
                  ├─ web_search → web_fallback → END
                  └─ retrieve → grade ─┬─ filtered docs? → generate → END
                                       ├─ retries left? → rewrite → retrieve  (loop)
                                       └─ else → web_fallback → END

Defaults: MAX_RETRIEVAL_RETRIES=2 · GRADER_RELEVANCE_THRESHOLD=0.5
```

Use when: answer quality matters and you can afford grading latency.

MODE 5.4 `decompose` — Parallel sub-queries

File: `src/graph/decompose_graph.py` · Map-reduce via LangGraph `Send` to `retrieve_sub` workers.

```
START → classify ─┬─ direct → direct_answer → END
                  ├─ web → web_search → END
                  └─ retrieve path → decompose
                                       │
                                       ├─ Send(sub_q1) → retrieve_sub ─┐
                                       ├─ Send(sub_q2) → retrieve_sub ─┼→ synthesize → END
                                       └─ Send(sub_qN) → retrieve_sub ─┘
```

Use when: "Compare X vs Y vs Z", multi-entity questions known upfront.

MODE 5.5 `multi_hop` — Sequential reasoning

File: `src/graph/multi_hop_graph.py` · Unlike decompose, hop *N+1* depends on findings from hop *N*.

```
START → classify ─┬─ direct → direct_answer → END
                  ├─ web → web_search → END
                  └─ analyze → retrieve_hop → reflect ─┬─ sufficient → synthesize → END
                                                       └─ next hop (until MAX_MULTI_HOP_STEPS)
```

Use when: "What fallback does CRAG use when retrieval fails?" — must learn CRAG first.

MODE 5.6 `tools` — Tool-calling agent

File: `src/graph/tools_graph.py` · Node `tools_agent` runs an internal loop (≤10 turns) with `llm.bind_tools(TOOLS)`.

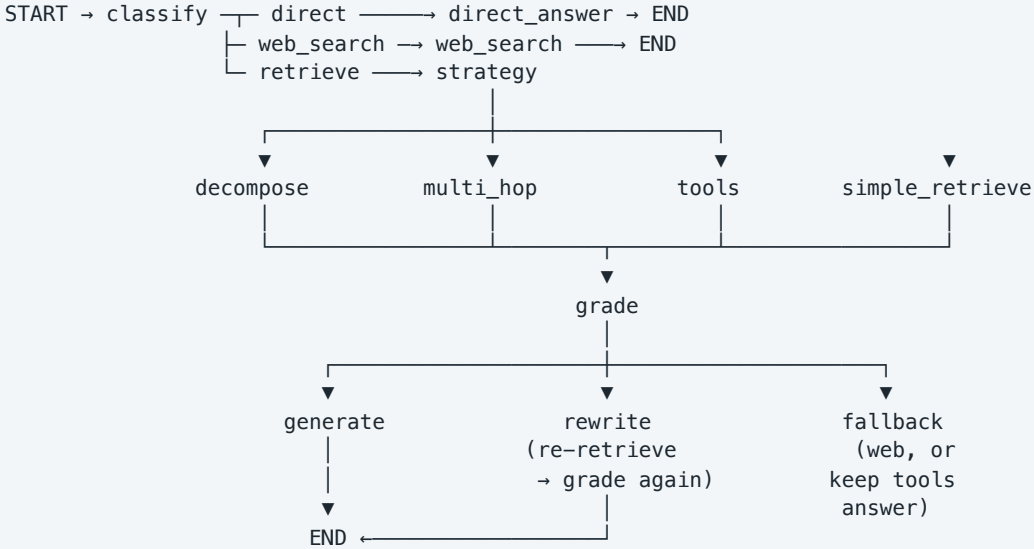
```
START → classify ─┬─ direct → direct_answer → END
                  ├─ web → web_search → END
                  └─ tools_agent ↻ (tool_calls) → END

TOOLS:
  • retrieve_docs(query)   → vector corpus
  • web_search(query)      → DuckDuckGo (circuit-breaker wrapped)
  • calculator(expr)       → AST-restricted arithmetic (no eval())
```

Use when: mixed modalities — math + corpus, or dynamic tool choice.

MODE 5.7 `agentic` — Full orchestrator (default)

File: `src/graph/agent_graph.py` · Combines routing, strategy selection, the four retrieval strategies (as subgraphs/nodes), and a CRAG-style grade/rewrite/fallback safety net on *every* retrieval path.



Stage	Decides	Options
`classify`	Do we need the corpus at all?	`direct` · `retrieve` · `web_search`
`strategy`	How should we retrieve?	`decompose` · `multi_hop` · `tools` · `simple`
`grade`	Is evidence good enough?	`generate` · `rewrite` · `fallback`

> **IMPORTANT IMPLEMENTATION DETAIL**
> Strategy nodes reuse the Phase 4–6 graphs with `skip_router=True` and token emission suppressed for nested generation, so the outer graph owns the final streamed answer. After `rewrite`, the edge is `rewrite → grade` (rewrite node re-retrieves internally). Tools with an empty grade but a prior tool answer keep that answer instead of forcing web fallback.

6. Decision chains the agents use

Graphs call small LangChain structured-output chains. These are the “brains” inside nodes.

Module	Function / chain	Produces
`agents/router.py`	`router_chain` / `classify_query`	`route` ∈ {direct, retrieve, web_search} + reason
`agents/grader.py`	`grade_documents`	Keep chunks with `relevant` and score ≥ threshold
`agents/query_rewriter.py`	`rewrite_query`	Rewritten search query + reason
`agents/decomposer.py`	`decompose_chain`	1–5 `sub_queries` + reasoning
`agents/multi_hop.py`	`analyze_chain`, `reflect_chain`	First hop plan; continue vs synthesize
`agents/orchestrator.py`	`choose_strategy`	`decompose` | `multi_hop` | `tools` | `simple`
`agents/followups.py`	`generate_follow_ups`	Up to 3 grounded follow-up questions

Strategy selection rubric (`agentic`)

Strategy	When the orchestrator should pick it	Example question
`decompose`	Comparisons / multiple distinct entities known upfront	Compare naive, advanced, and modular RAG
`multi_hop`	Answer needs fact A before you can search for fact B	What fallback does CRAG use when retrieval fails?
`tools`	Mix of retrieval + math/web/function calling	What is Self-RAG and what is 20×30?
`simple`	Single-topic factual lookup	What is Self-RAG?

7. Ingestion & retrieval pipeline

7.1 Ingestion (before any query)

```
PDF (PyMuPDF)
  → cleanse (headers/footers/boilerplate/irrelevant sections)
  → chunk
       • section_parent_child (default): TOC/heading parents → child splits
       • fixed: RecursiveCharacterTextSplitter
  → save parents → data/parent_store.json
  → embed (OpenAI) → upsert Chroma (content-hash chunk_id)
  → invalidate BM25 cache + flush Redis answer cache
```

Command: `python -m src.ingestion.ingest --source data/sample_docs` · Use `--force` to rebuild. Compose uses `CHROMA_MODE=http`; local dev defaults to persistent SQLite under `data/chroma_db`.

7.2 Retrieval (shared by all modes)

```
query
  → over-fetch candidate_k
       hybrid (default): dense kNN + BM25 → Reciprocal Rank Fusion
       or mmr / similarity
  → optional cross-encoder rerank
       nvidia (NeMo API, circuit breaker "nvidia_rerank")
       or flashrank (local ONNX)
  → truncate to top_k
  → optional expand_children_to_parents (cap parent_max_chars)
  → documents for grading / generation / citations
```

Knob	Default role
`RETRIEVAL_CANDIDATE_K=20`	Over-fetch pool before rerank
`RETRIEVAL_TOP_K=6`	Final chunks to the LLM
`RETRIEVAL_SEARCH_TYPE=hybrid`	Dense + BM25 + RRF
`RERANK_PROVIDER=nvidia`	Cross-encoder rescoring
`EXPAND_TO_PARENT=true`	Return parent sections for context

8. Generation, citations, follow-ups, streaming

8.1 Generation chains

LCEL chains in `src/chains/generation.py`: `rag_chain`, `direct_chain`, `synthesis_chain`, `web_search_chain`. All use `get_llm()` from `src/llm.py`.

8.2 LLM client

- **Primary:** OpenAI `ChatOpenAI` with hard `timeout`, `max_retries`, `max_tokens`.
- **Fallback:** if `LLM_FALLBACK_ENABLED` and `GROQ_API_KEY` are set, failed primary calls retry on Groq via LangChain `with_fallbacks`.

- **Embeddings** always stay on OpenAI (even during chat fallback).

8.3 Citations

`src/retrieval/citations.py` builds `Citation` objects: `index`, `chunk_id`, `source`, `page`, `section`, `snippet`, `score`. The frontend renders them; evals use them for grounding checks.

8.4 Follow-ups

After every successful run, `generate_follow_ups` proposes up to three questions grounded in retrieved snippets (or the answer text as fallback). Failures are swallowed so follow-up generation never fails a query.

8.5 Streaming internals

`src/streaming.py` provides a ContextVar emitter, `stream_text` / `stream_llm_message` for tokens, and `run_graph_streaming` for step events as LangGraph nodes complete.

9. Production envelope

Everything outside the graphs that makes the system safe to expose.

Concern	Mechanism	Where
Auth	API key, constant-time compare; mandatory if `ENVIRONMENT=production`	`api/security.py`
Per-client rate limit	Sliding window; Redis when `RATE_LIMIT_BACKEND=auto|redis`	`api/rate_limit.py`
Process cost budget	QPM + tokens/min + tokens/hour	`guardrails.py`
Privacy	PII/PHI detect; redact/block output	`privacy.py`
Answer cache	Redis key `rag:v1:{mode}:{hash}` ; skip when memory changes answer; flush on re-ingest	`cache/redis_cache.py`
Idempotency	`Idempotency-Key` on sync `/query`	server + Redis
Circuit breakers	Fail-fast for `nvidia_rerank` and `web_search`	`resilience/circuit_breaker.py`
Node output gates	Quarantine bad tool/node outputs; abort critical path (`error_code=node_gate_abort`)	`resilience/node_gate.py`
Metrics	Prometheus counters/histograms at `GET /metrics`	`api/metrics.py`
Tracing	Optional LangSmith via `bootstrap.py`	`observability.py`
Memory	Compact packing; optional Supabase persistence	`memory/*`

> **MULTI-WORKER RULE**
> Before raising `API_WORKERS` above 1, run Redis and set `RATE_LIMIT_BACKEND=redis` (compose already does). In-memory limits are per-process and under-count under load balancing.

9.1 Evaluation

- **Golden retrieval:** `data/eval/golden_qa.json` + `python -m src.evaluation.retrieval_metrics` (use `--offline` in CI).
- **LLM-as-judge:** `python -m src.evaluation.evaluate_all_modes` → faithfulness / relevance / context precision.

10. API surface & deploy topology

10.1 HTTP API

Method	Path	Auth	Purpose
GET	`/health`	no	Liveness
GET	`/health/ready`	no	Readiness (Chroma, keys, Redis, …)
GET	`/metrics`	no	Prometheus scrape
GET	`/modes`	yes*	Mode labels
POST	`/query`	yes*	Sync JSON answer
POST	`/query/stream`	yes*	SSE progressive answer

* Auth enforced when `REQUIRE_API_KEY=true` or `ENVIRONMENT=production`.

```
POST /query
{
  "question": "What is Self-RAG?",
  "mode": "agentic",          // default
  "session_id": "optional-id",
  "use_memory": true,
  "chat_history": [{"role":"user","content":"..."}, {"role":"assistant","content":"..."}]
}
```

10.2 Docker Compose services

Service	Port	Role
`redis`	6379	Cache, shared rate limits, idempotency
`chroma`	8001→8000	Vector store (HTTP)
`agentic-rag`	8000	FastAPI app
`frontend`	8080	nginx + React; proxies `/api`; can inject API key
`prometheus`	9090	Optional metrics store
`grafana`	3000	Optional “Agentic RAG Overview” dashboard

```
# Core stack
cp .env.production.example .env.production   # fill secrets
docker compose up -d

# Optional observability
docker compose up prometheus grafana -d
```

11. Configuration map

All settings load via pydantic-settings in `src/config.py` from environment / `.env`.

Group	Key variables
LLM	`OPENAI_API_KEY`, `OPENAI_MODEL`, `OPENAI_EMBEDDING_MODEL`, timeouts/retries/ `MAX_OUTPUT_TOKENS`
Fallback	`LLM_FALLBACK_ENABLED`, `GROQ_API_KEY`, `GROQ_MODEL`
Vector store	`CHROMA_MODE` (persistent|http), host/port, `COLLECTION_NAME`
Chunking	`CHUNKING_STRATEGY`, child/parent sizes, `EXPAND_TO_PARENT`, `PARENT_STORE_PATH`
Cleanse	`CLEANSE_ENABLED` and `CLEANSE_*` toggles
Retrieval	`RETRIEVAL_TOP_K`, `RETRIEVAL_CANDIDATE_K`, `RETRIEVAL_SEARCH_TYPE`, RRF/MMR knobs
Rerank	`RERANK_ENABLED`, `RERANK_PROVIDER`, `RERANK_MODEL`, `NVIDIA_API_KEY`
Agent	`MAX_RETRIEVAL_RETRIES`, `GRADER_RELEVANCE_THRESHOLD`, `MAX_MULTI_HOP_STEPS`
API / security	`ENVIRONMENT`, `API_KEY`, `REQUIRE_API_KEY`, `CORS_ORIGINS`, `TRUSTED_HOSTS`
Rate / cost	`MAX_QUERIES_PER_MINUTE*`, `MAX_TOKENS_*`, `RATE_LIMIT_BACKEND`
Cache / Redis	`CACHE_ENABLED`, `REDIS_URL`, `CACHE_TTL_SECONDS`
Resilience	`CIRCUIT_BREAKER_*`, `REQUEST_TIMEOUT_SECONDS`
Memory	`MEMORY_*`, `SUPABASE_URL`, `SUPABASE_KEY`
Observability	`LANGSMITH_*`

12. File ownership quick reference

Path	Owns
`src/runner.py`	Mode dispatch, lifecycle, cache/memory/follow-ups/streaming orchestration
`src/graph/*_graph.py`	Per-mode LangGraph topologies + `ask_*` entrypoints
`src/rag/baseline.py`	Phase 1 fixed pipeline
`src/agents/*`	Structured LLM decision chains
`src/retrieval/*`	Hybrid search, rerank, citations
`src/ingestion/*`	Cleanse, chunk, parent store, Chroma upsert
`src/tools/all_tools.py`	`retrieve_docs` , `web_search` , `calculator`
`src/llm.py`	OpenAI primary + optional Groq fallback
`src/streaming.py`	SSE emitter, tokens, graph step streaming
`src/cache/redis_cache.py`	Answer cache + idempotency helpers
`src/resilience/circuit_breaker.py`	Named breakers
`src/api/server.py`	HTTP routes and response models
`src/api/metrics.py`	Prometheus instrumentation
`frontend/`	React chat UI (SSE, citations, follow-ups, traces)
`monitoring/`	Prometheus scrape config + Grafana dashboard
`data/eval/golden_qa.json`	Retrieval golden set

13. Worked traces

Re-run these with `-v` to see `steps` fill in live.

13.1 Simple fact — `baseline` vs `crag`

```
python -m src.cli ask "What is Self-RAG?" --mode baseline -v
python -m src.cli ask "What is Self-RAG?" --mode crag -v
```

> **WHAT TO NOTICE**
> Baseline: one retrieve → generate. CRAG: retrieve → grade each chunk → maybe rewrite → generate (or web fallback if the corpus is weak).

13.2 Comparison — `decompose`

```
python -m src.cli ask "Compare naive RAG, advanced RAG, and modular RAG" --mode decompose -v
```

> **WHAT TO NOTICE**
> `sub_queries` appear; parallel `retrieve_sub` workers; one synthesis.

13.3 Sequential dependence — `multi_hop`

```
python -m src.cli ask "What fallback does CRAG use when retrieval fails?" --mode multi_hop -v
```

> **WHAT TO NOTICE**
> Hop 1 finds CRAG; reflect builds hop 2 query about fallback; synthesize.

13.4 Full orchestrator — `agentic`

```
python -m src.cli ask "Compare RAG vs Agentic RAG; what is Self-RAG grading?" --mode agentic -v
```

> **WHAT TO NOTICE**
> Steps show `Router → retrieve`, then `Strategy: decompose|…`, then grade, then generate. Same question through baseline will be shallower.

13.5 HTTP smoke

```
uvicorn src.api.server:app --reload --port 8000

curl -s http://localhost:8000/health
curl -s -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"question":"What is Self-RAG?","mode":"agentic"}' | jq '.answer,.steps,.follow_ups'

curl -N http://localhost:8000/query/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"What is Self-RAG?","mode":"agentic"}'
```

Agentic RAG — System Reference Manual · August 2026

Canonical code entry points: `src/runner.py`, `src/graph/`, `src/api/server.py`. Companion docs: `README.md`, `docs/ROADMAP.md`, `docs/PRODUCTION.md`, `docs/GUARDRAILS.md`, `docs/LANGCHAIN_STACK.md`.

If code and this manual diverge, trust the code — then update this file.